Escuela de Ingeniería Electrónica.

Taller de Diseño Digital.

Documentación laboratorio #2

Tema:

Microcontrolador

Profesor:

Kaled Alfaro Badilla.

Estudiantes:

María Paulina Camacho Herrera – 2021093601

Kendall Jesús Barrantes Chaves-2021442238

Luis Daniel Zumbado Sáenz - 2018176234

José Pablo Herrera Vagas - 2023029239

I Semestre 2026

En el presente expone de manera sistemática el diseño, desarrollo e implementación de un sistema digital correspondiente al segundo proyecto del curso de taller de diseño digital. La arquitectura del sistema se enfoca en RISC-V, específicamente el set de instrucciones de RV321, que se encarga de representar la base de procesamiento de enteros de 32 bits para ofrecer eficiencia y modularidad.

A continuación, se dará una descripción detallada de cada uno de los módulos desarrollados individualmente que permiten el funcionamiento correcto del sistema.

Netsys_4_DDR.xdc

Corresponde al archivo de constraints de Xilinx (XDC). Tiene como propósito de asignar los pines físicos de la FPGA a las señales definidas en el módulo top.

PicoRV32

PicoRV32 es un procesador RISC-V de 32 bits completo, extremadamente pequeño y optimizado, diseñado para ser implementado en FPGA y ASICs con recursos limitados.

El código representa un procesador single – issue, in – order, pipeline de 1 etapa. Su principal objetivo es lograr el menor tamaño posible manteniendo una compatibilidad con el estándar RISC-V, permitiendo al usuario activar o desactivar características según las necesidades.

Componentes internos principales:

Banco de registros: se implementan ya sea los 16 o 32 registros del RISC-V siempre es cero.

Máquina de estados: controla todo el flujo del procesador de estados como, por ejemplo:

- Fetch (buscar instrucción).
- Ld_rs1/ld_rs2 (leer operandos).
- Exec (ejecutar).
- Shift (desplazamientos).
- Ldmem / stmem (lectura / escritura de memoria)

Decodificador de instrucciones:

Soporta tanto instrucciones de 32 bits (RV32I) como 16 bits (Compressed ISA – extensión C).

Decodifica todos los tipos de instrucciones: LUI, AUIPC, JAL, branches, loads, stores, operaciones ALU, system, etc.

Genera señales individuales (instr_add, instr_lw, instr_beq) para facilitar el control.

ALU (unidad aritmeticológica):

Realiza sumas, restas, AND, OR, XOR, comparaciones y shifts.

Puede trabajar en uno o dos ciclos según el parámetro.

Gestor de memoria:

Maneja accesos alineados y no alineados.

Soporta prefetch de instrucciones.

Maneja especialmente bien las instrucciones comprimidas (es capaz de leer palabras de 32 bits y extraer instrucciones de 16 bits).

Unidad de control de interrupciones:

Soporta interrupciones personalizadas, temporiazdor, ebreak.

Incluye instrucciones especiales como retirq, waitirq, maskirq.

Co-procesador PCPI:

Permite conectar multiplicadores, divisores o aceleradores personalizados.

Incluye implementaciones internas para multiplicación y división iterativa.

Flujo de ejecución típico:

→ Se busca una instrucción desde la memoria.
→ Se decodifica (normal o comprimida).
→ Se leen los registros fuente.
→ Se ejecutan las operaciones (ALU, branch, load/store).
→ Se escribe el resultado si es necesario.
→ Se actualiza el program counter (reg_pc).
→ Se verifica si hay interrupciones pendientes.

Módulos auxiliares incluidos:

Picorv32_pcpi_mul y picorv32_pcpi_fast_mul: multiplicadores.

Picorv32_pcpi_div: divisor/resto.

Picorv32_regs: banco de registros por defecto.

Picorv32_axi y picorv32_wb: wrappers para conectar el procesador a buses estándar (AXI4-Lite y Wishbone).

Módulo de trabajo top.

Este módulo corresponde a un archivo de nivel superior (top) de un sistema embebido basado en el procesador PicoRV32. Su función es integrar todos los componentes del sistema en un solo diseño para ser sintetizado en una FPGA

Corresponde a un sistema System-on-Chip (SoC) que contiene:

→ Un procesador RISC-V(PicoRV32).
→ Memoria (RAM + GPIO).
→ Un periférico UART (para comunicación serie).
→ Bus de interconexión para que el procesador se comunique con los periféricos.

Módulo UART_peripheral

Este módulo implementa un periférico UART (transmisor y receptor en serie) para el procesador PicoRV32. Funciona como un dispositivo de memoria mapeada (Memory-Mapped I/O), es decir, el procesador controla escribiendo y leyendo en direcciones específicas de memoria, como si fueran registros normales.

Para transmitir: el procesador escribe un byte en 0x2010.

Para recibir: el procesador lee de 0x2014.

Para saber el estado: lee 0x2018. No usa interrupciones, lo que lo vuelve más simple y adecuado para principiantes. Constituye un periférico ligero, de bajo uso de recursos y perfectamente integrado con el bus del sistema que se observó en el módulo top.

Módulo UART_RX

Este módulo implementa un receptor UART. Su función es recibir datos serie desde el pin rx y convertirlos en un byte paralelo de 8 bits, avisando al periférico cuando un byte completo ha sido recibido.

Consiste en un diseño muy básico, sin oversampling, sin paridad y sin bit de stop verificado.

Señales principales:

Señal	Dirección	Descripción
Clk	Entrada	Reloj del sistema

Rst	Entrada	Reset (activo alto)
Rx	Entrada	Línea serie de recepción
Donerx	Salida	Pulso que indica que se recibió un byte completo.
Doutrx	Salida	Byte recibido (8 bits)

Máquina de estados

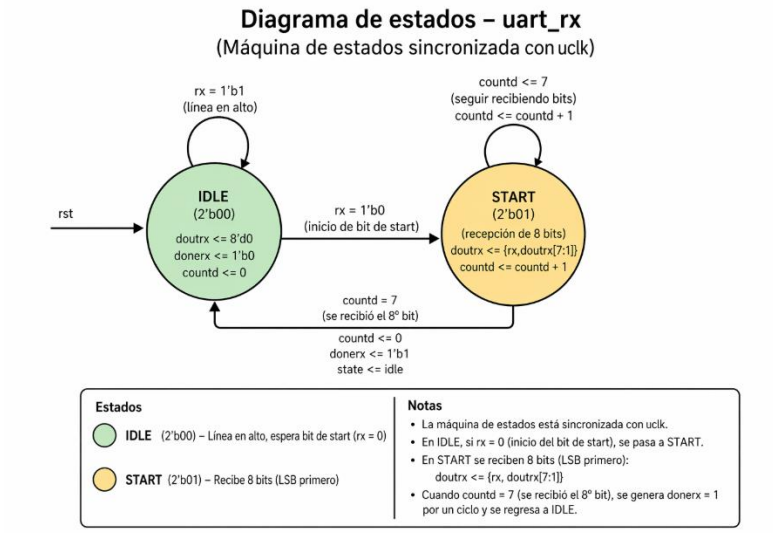

Fig.1. Diagrama de estados correspondiente a la máquina de estados del módulo UART_RX

Como se observa en la figura 1, la máquina de estados se encarga de controlar la recepción serial de datos en el sistema UART, que se encuentra sincronizado con el reloj UCLK. Se presentan dos estados:

→ Idle (reposo)

→ Start (recepción de datos)

Diagrama de bloques:

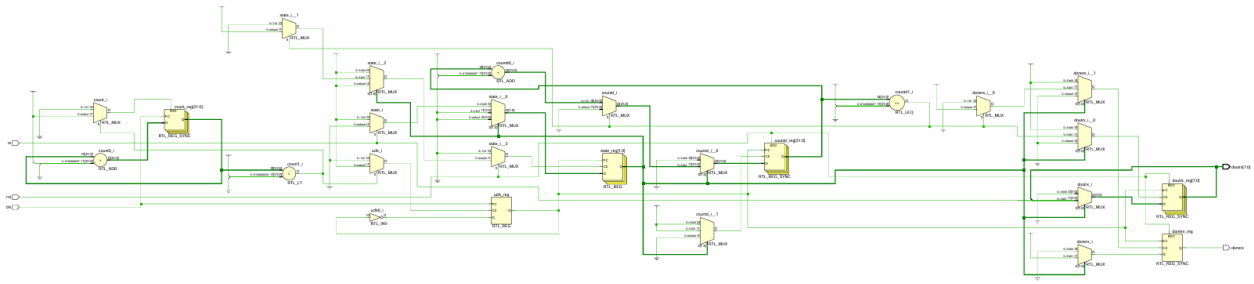

Fig.2. Diagrama de bloques correspondiente al módulo UART_RX.

Módulo UART_TX

Este módulo implementa un transmisor UART(TX) básico. Su función consiste en tomar un byte paralelo de 8 bits cuando se activa la señal, y transmitirlo en serie por el pin tx a la velocidad configurada.

Puertos del módulo

Señal	Dirección	Descripción
Clk	Entrada	Reloj del sistema
Rst	Entrada	Reset (activo alto)
Newd	Entrada	Pulso que indica el nuevo dato para transmitir
Tx_data	Entrada	Byte de 8 bits para transmitir
Tx	Salida	Línea serie de transmisión
Donetx	Salida	Pulso que indica que la transmisión ha terminado

Máquina de estados:

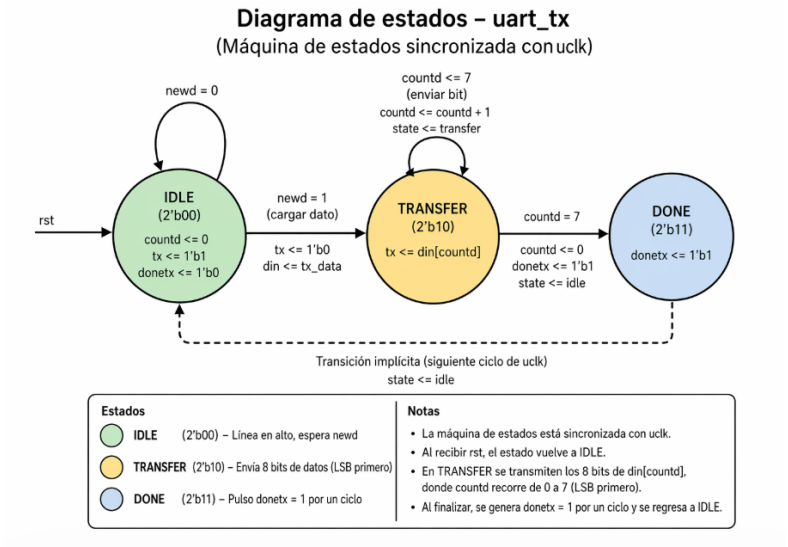

Fig.3. Diagrama de estados correspondiente a la máquina de estados del módulo UART_TX.

Como se observa en la figura 3, la máquina de estados se encarga de controlas la transmisión serial de datos por medio de UART

Diagrama de bloques:

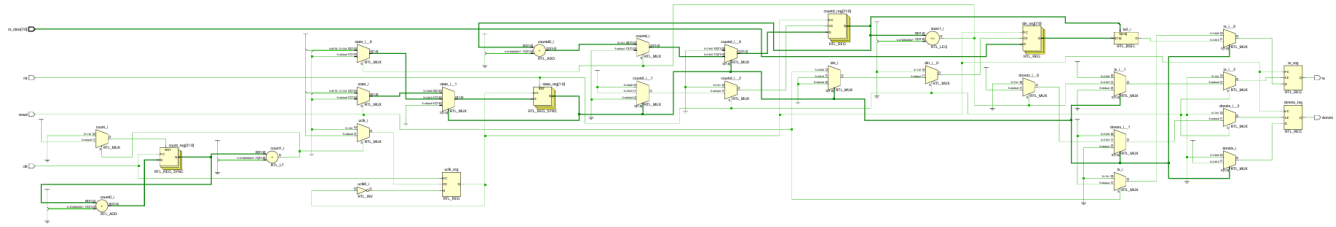

Fig.4. Diagrama de bloques correspondiente al módulo UART_RX.

Módulo Clock_gen

Este módulo genera un reloj de sistema usando un bucle de enganche (PPL) que acondicionara el sistema dentro de la FPGA.

Puertos del Módulo

Señal	Dirección	Descripción
Clk_in	Entrada	Reloj del sistema
rst	Entrada	Reset manual
Clk_out	Salida	Reloj procesado
locked	Salida	PPL estable

Internamente este módulo instancia un bloque llamado clk_wlz_0 el cual se genera automáticamente y corresponde al PPL cuya función es procesar el reloj de entrada para mejorar su calidad y reducir el jitter. La señal locked indica que el bucle de enganche a logrado sincronizarse correctamente con el reloj de entrada, mientras la señal se encuentre activa, el reloj de salida es confiable.

Modulo memory

Este módulo funciona como un bloque central que combina memoria y periféricos dentro del sistema, este recibe peticiones del CPU y decide según su dirección donde dirigirlas, por lo que actúa como un bloque multifuncional que integra memoria de programa, memoria de datos y periféricos básicos que sean accesibles mediante direcciones desde el procesador.

Puertos del modulo

Señal	Dirección	Descripción
clk	Entrada	Reloj del sistema

Valid	Entrada	Petición del CPU
ready	Salida	Indica cierre del sistema
addr	Entrada	Dirección de Memoria
wdata	Entrada	Dato para escribir
wstrb	Entrada	Habilita escritura
rdata	Salida	Dato leído desde memoria
Led_out	Salida	Control de LEDS
sw_in	Entrada	Estado de switches físicos

El módulo incluye una memoria ROM y memoria RAM, LA ROM se inicializa desde el archivo firmware.hex que contiene el programa que ejecuta el procesador, la RAM permite lectura y escritura y almacena datos durante la ejecución del programa. Este módulo también controla periféricos simples ya que tiene un registro interno que maneja los LEDS de salida y permite leer el estado de los switches, el módulo incluye lógica de decodificación de direcciones para determinar a qué recurso se está accediendo.

Decodificación de direcciones

→ ROM: Devuelve una instrucción.

→ RAM: Permite leer y escribir datos.

→ Leds: Escribe o devuelve su estado.

→ Switches: devuelve su valor.

Diagrama de bloques:

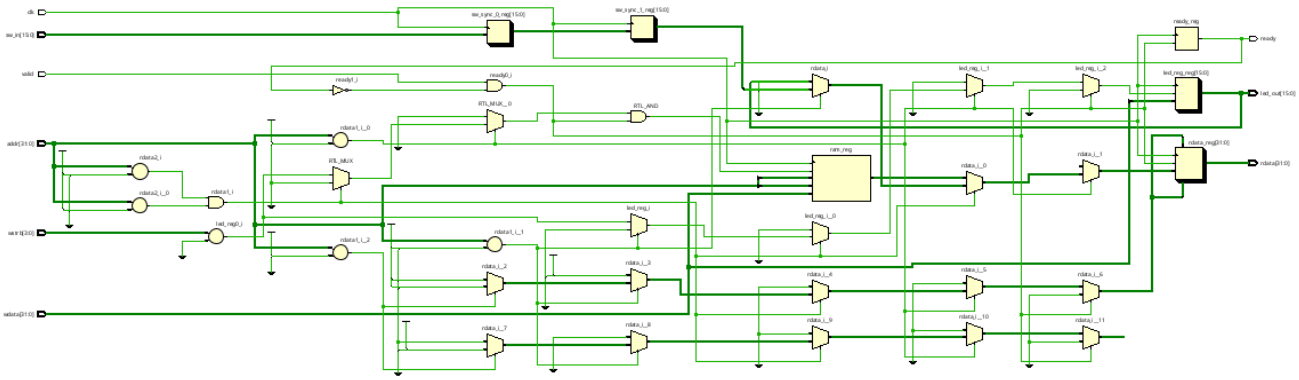

Fig.5. Diagrama de bloques correspondiente al módulo de memoria.

Módulo bus interconnect

Este módulo es el routwer central entre el CPU y todos los bloques, para su funcionamiento primeramente el CPU genera una transacción donde el bus decodifica la dirección y selecciona un destino (ROM, RAM, IO, LEDS o switch), posteriormente envía la solicitud solo al bloque seleccionado y multiplexa la respuesta de ese bloque hacia el CPU.

Las entradas y salidas del módulo van a depender de cada bloque:

CPU

Señal	Dirección
mem_valid	Entrada
mem_ready	Salida
mem_addr	Entrada
mem_wdata	Entrada
mem_wstrb	Entrada
mem_rdata	Salida

ROM

Señal	Dirección
rom_valid	Salida
rom_ready	Entrada
rom_addr	Salida
rom_rdata	Entrada

RAM

Señal	Dirección
ram_valid	Salida
ram_ready	Entrada
ram_addr	Salida
ram_wdata	Salida
ram_wstrb	Salida
ram_rdata	Entrada

IO (LEDS y swtich)

Señal	Dirección
io_valid	Salida
io_ready	Entrada
io_addr	Salida
io_data	Salida
io_wstrb	Salida
io_rdata	Entrada

UART

Señal	Dirección
uart_valid	Salida
uart_ready	Entrada
uart_addr	Salida
uart_wdata	Salida
uart_wstrb	Salida
uart_rdata	Entrada

Cada uno de los bloques procesa la solicitud y responde mediante la señal ready y el dato de salida rdata, este módulo selecciona cual de estas respuestas debe regresar al procesador, esta función la realiza mediante un multiplexor que dependiendo de la dirección conecta la salida del módulo hacia las señales del CPU.

Diagrama de bloques:

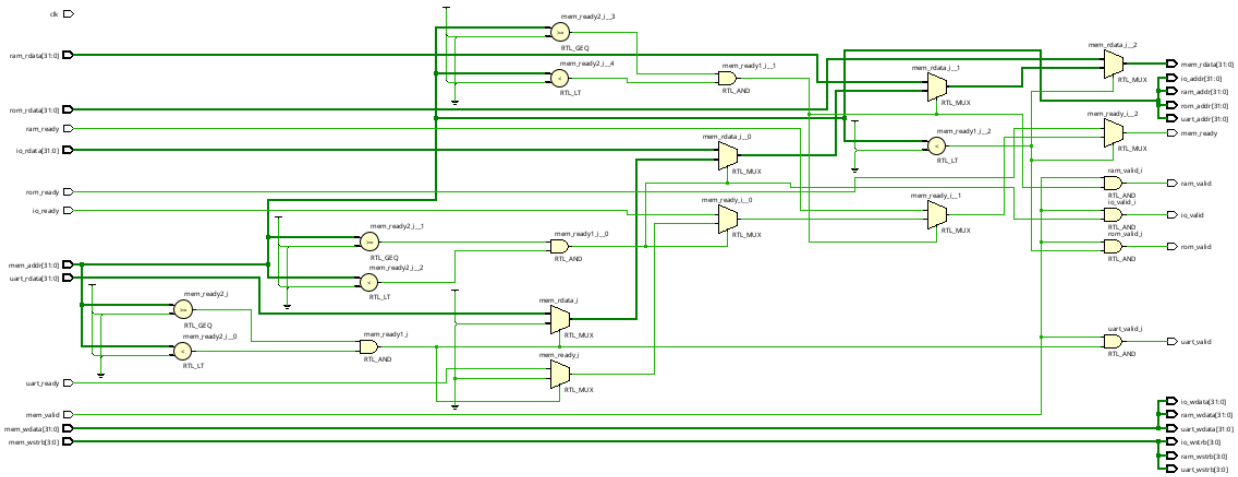

Fig.6. Diagrama de bloques correspondiente al módulo bus interconnect.

Picorv32_tb

Esté es el código completo que se utiliza para simular el procesador con interfaz AX14 cuyo propósito en verificar que el procesador ejecute correctamente un programa cargado en memoria, en este caso firmware.

Primeramente, se genera el módulo testbench para el entorno de simulación donde se produce la señal de reloj y se define un tiempo máximo de ejecución y opcionalmente puede generar archivos de trazas para depuración.

Dentro del módulo se instancia el sistema principal picorv32_wrapper que conecta todos los componentes del sistema, el cual, incluye el procesador Picorv32 y la memoria simulada axi4_memory, en este módulo se carga el firmware desde un archivo .hex a la memoria, que contiene el programa que ejecutara el procesador. Durante la simulación se cuenta el número de ciclos y cuando el procesador termina si verifica si se superaron las pruebas.

El módulo axi4_memory implementa una memoria RAM que maneja tanto lecturas como escrituras siguiendo el handshake del protocolo, incluye lógica para leer datos desde memoria cuando el procesador lo solicita, escribir datos, detectar accesos fuera de rango y simular comportamiento realista con retrasos aleatorios en las transacciones AXI.

Esta memoria contiene direcciones especiales:

- En la dirección 0x1000_0000, cualquier dato escrito se imprime como salida (simula una consola).

- En la dirección 0x2000_0000, si se escribe el valor 123456789, se marca que las pruebas pasaron (tests_passed = 1).

El flujo general es: la simulación inicia, se carga el firmware en memoria, el procesador ejecuta instrucciones accediendo a la memoria mediante AXI, y al finalizar activa la señal trap. En ese momento, el sistema verifica si el programa indicó éxito y muestra el resultado.